#IA para Devs

- **O que é Inteligência Artificial**

Aplicar matemática e ciência da computação para que as máquinas possam fazer, agir e aprender da mesma maneira que os humanos.

- **Subáreas da IA**

↱ tudo em uma arquitetura unificada.

Strong AI → área em que uma IA tem um contexto generalista global e totalmente multimodal. Área ainda não alcançada, mesmo tendo grandes modelos LLM.

↳ Chat GPT por exemplo arquiteta múltiplas arquiteturas de modelo, em diferentes modalidades, para parecer um Strong AI.

↳ AGI (Artifical General Intelligence)

Weak AI/Narrow AI → modelos focados em apenas uma área, onde os modelos canônicos (2026) se encontra. Pode ser dividido em subáreas.

↳ (A) Symbolic AI: abordagem em que o computador resolve problemas usando regras lógicas e (God Old Fashioned AI) símbolos (como palavras ou conceitos), em vez de dados numéricos e estatísticos (e.g Robotic Process Automation - RPA)

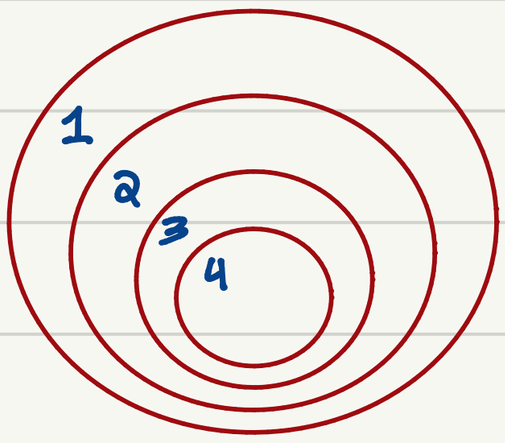
↳ Machine Learning: aprendizado com base em padrões

↳ Classical ML: evolução básica da estatística, previsão e classificação.

↳ Deep Learning: redes neurais conectadas em múltiplas camadas, aprendendo padrões mais finos.

↳ Reinforcement Learning: aprendizado com base em erros, acertos e recompensas.

- **Subdivisão Clássica:**



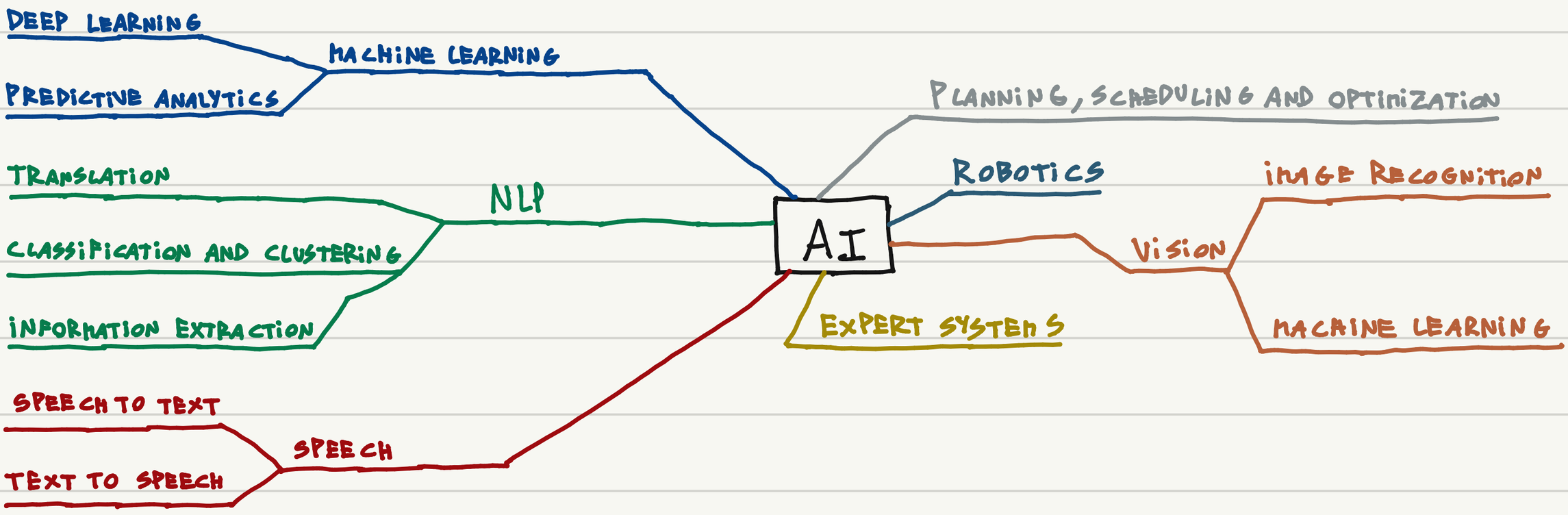
1 - Artificial Inteligence (e.g. RPA).

2 - Machine Learning (e.g. Linear Regression, Decision Tree, Randon Forest, XGBoost).

3 - Deep Learning (e.g. CNN's, RNN's, LSTM's, GAN's).

4 - Generative AI (e.g. LLM's, LVM's)

- Áreas de Aplicação de IA



- AI Applying - SaaS

Modelos genéricos para consumir como serviços (e.g. Azure, AWS, Google Cloud, IBM Cloud, OpenAI).

e.g. User —(Question)→ Chat GPT (Frontend) —(Request (Query + instructions))→ GPT (Modelo LLM)
User ←(Response)— Chat GPT (Frontend) ←(Response)— GPT (Modelo LLM)

➡ É possível configurar nesse SaaS como os modelos vão agir como especialista.

LangFlow: criar especialistas de uma forma mais robusta, conectando APIs de IA e, ao final, o serviço do especialista pode ser acessado com uma API.

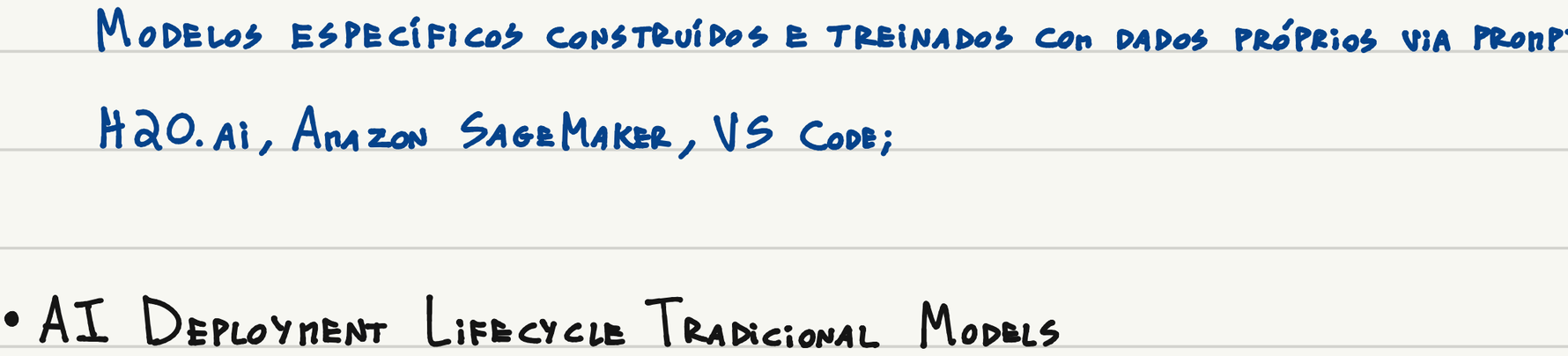
- AI Creating

Modelos específicos construídos e treinados com dados próprios via prompt (e.g. Jupyter, Databricks, H2O.ai, Amazon SageMaker, VS Code;

- AI Deployment Lifecycle Tradicional Models

(A) Modelos específicos e treinados via programação. +Time to value

(B) Modelos Genéricos



- Acesso de serviços via código Python (e.g. API da OpenAI)

- Chatbots surgem para suprir necessidades de altas demandas de comunicação, principalmente no relacionamento entre cliente e empresa.

↑ Stemming

- Lematização: técnica de pré-processamento com NLP que reduz palavras flexionadas à sua forma base ou de dicionário, chamada de lema.

- Word Embedding: tradução de palavras em vetores que, numericamente se arranjam em um espaço por similaridade entre si.

- Tipos de Chatbot - Funcionalidade

Chatbot do tipo:

Question and Answer

(A) Pergunta e resposta.

(B) Informativos.

(C) FAQ.

(D) Objetivo de atender e não executar.

Chatbot do tipo:

Transacional

(A) Pergunta e ação.

(B) Resolutivos.

(C) Integração com sistemas (API's, RPA).

(D) Objetivo de executar ações pelo usuário.

- Tipos de Chatbots - Interação

(A) Menu-Based Chatbots.

(B) AI-Powered Chatbots.

(C) Generative AI Chatbots.

↑ (Q&A e transacionais)

- IBM Cloud - Watson Assistant → Cobra por uso da API.

↳ (A) Opção baseada em menu:

Cria uma sequência de passos e o usuário vai interagindo (step 1 → step 2). Tudo programado via interface. Configura condições caso o usuário fuja do que deve ser descrito.

(B) Opção baseada em processamento de linguagem natural:

É definido ações, as quais são interpretadas baseado nas falas do usuário, que levam os passos a acontecer. As ações são definidas por frases, deve-se evitar definir ações apenas por palavras-chave.

↳ Não deve ser confundido com respostas puramente de IA generativa, pois cada ação deve levar a uma automação de processo bem definida, uma IA puramente generativa irá gerar respostas novas.

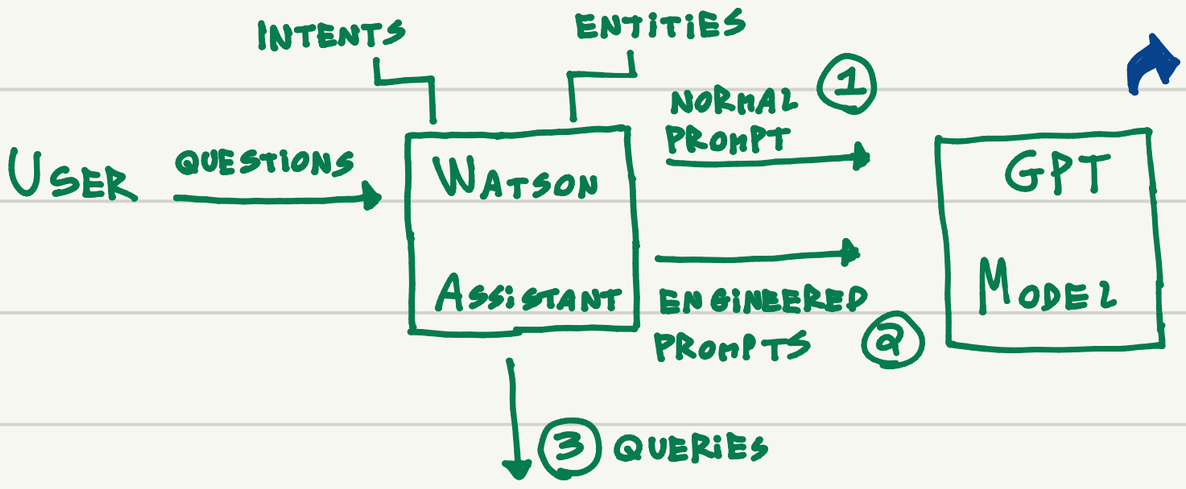
↳ OUTRAS OPÇÕES: DIALOGFLOW E DIALOGFLOW CX, FUNCIONALIDADE BASEADA EM AÇÕES COM PROCESSAMENTO DE LINGUAGEM NATURAL, SEMELHANTE AO IBM WATSON.

↳ CHAT GPT PELO USO DA API PODE SER CONFIGURADO PARA RESPONDER COM CHATBOT COM REGRAS BEM DEFINIDAS.

↳ NA AULA É MONTADO COM UMA INTERFACE DO STREAMLIT, AS INSTRUÇÕES DE INTERAÇÃO SÃO PASSADOS POR PROMPTS, E FICA MAIS CARO PELA COBRANÇA COM TOKENS.

↳ PREENCHER OS CAMPOS DE PERSONALIZAÇÃO PARA USO COMO MOSTRADO NO TEMPLATE DA OPENAI.

- O USO DO IBM WATSON TORNA UMA OPÇÃO MENOS PERSONIFICADA, PORÉM MAIS EFICAZ E DIRETA. ALÉM DISSO, OUTRAS PERGUNTAS QUE FOGEM DOS MENUS PODERIAM SER INTERESSANTES DE SEREM RESPONDIDAS. DESSE MODO, É POSSÍVEL, POR EXEMPLO, CONECTAR O IBM WATSON AO CHAT GPT.



↳ e.g. PERGUNTAS QUE CAEM SEM AÇÕES.

- TEXT COMPLETATION MODEL.
- CHAT MODEL.

↳ NO REPOSITÓRIO DO IBM WATSON DO GITHUB, TEM OS TEMPLATES .JSON PARA AUXILIAR A INTEGRAÇÃO DA EXTENSÃO.

DESCREVEM APIs DE FORMA PADRONIZADA, INCLUINDO ENDPOINTS, PARÂMETROS E ESQUEMA DE DADOS.

↳ CONFIGURAR LIMITAÇÕES PARA O GPT NÃO SAIR RESPONDENDO COISAS DESNECESSÁRIAS.

↳ MESMO ASSIM HÁ GASTOS ADICIONAIS DE TOKENS PARA VERIFICAR ISSO E O CHATGPT DIZER: "NÃO POSSO RESPONDER ISSO".

- **LLM's - LARGE LANGUAGE MODELS**

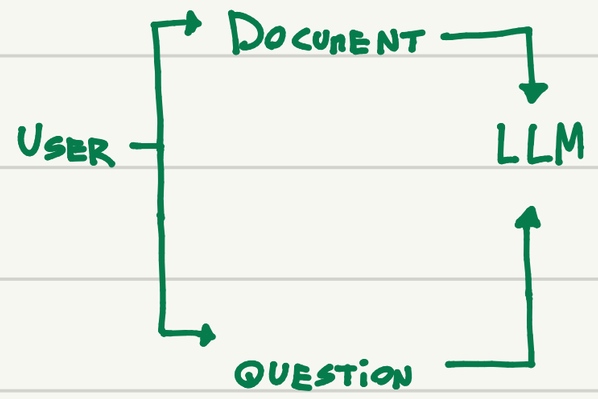
MODELOS DE CONHECIMENTO GERAL TREINADOS COM UMA VASTA QUANTIDADE DE DADOS. APRENDER PADRÃO LINGUÍSTICO.

REALIZAR VÁRIAS TAREFAS COMO TRADUÇÃO, CLASSIFICAÇÃO, GERAÇÃO DE TEXTO, ETC.

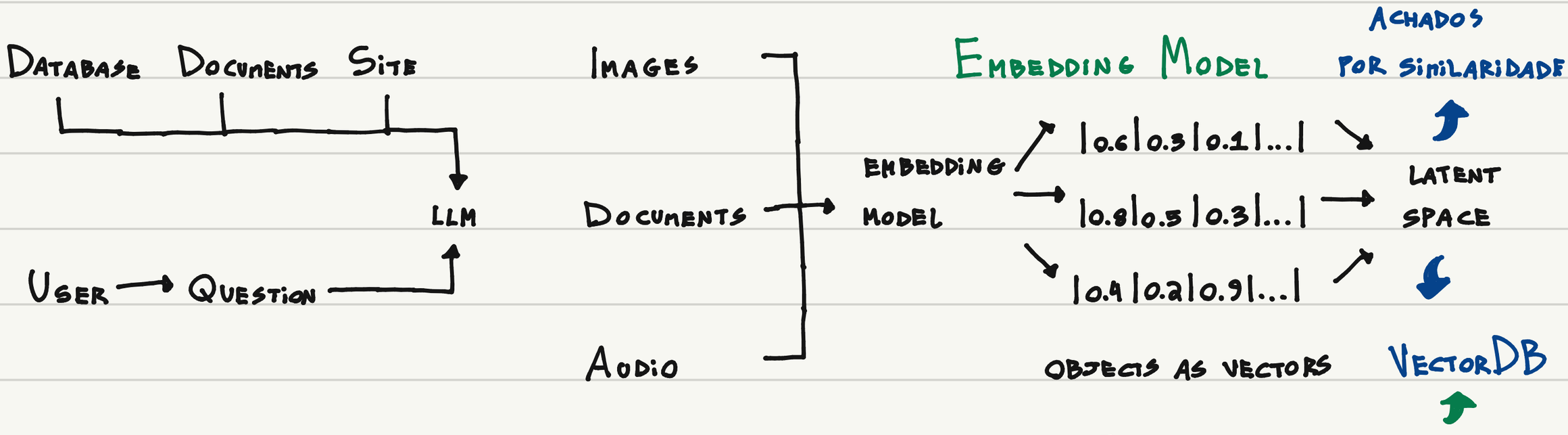
- LIMITAÇÃO DOS LLM's

  - LIMITADO A BASE (GIGANTESCA) DE TREINAMENTO. → SOLUÇÃO: PESQUISA ONLINE.
  - DADOS PÚBLICOS (OU NÃO) GERAIS.
  - NÃO ATENDE NECESSIDADES ESPECÍFICAS DE CADA EMPRESA/PESSOA.

↳ ADIÇÃO DE CONTEXTO ESPECÍFICO

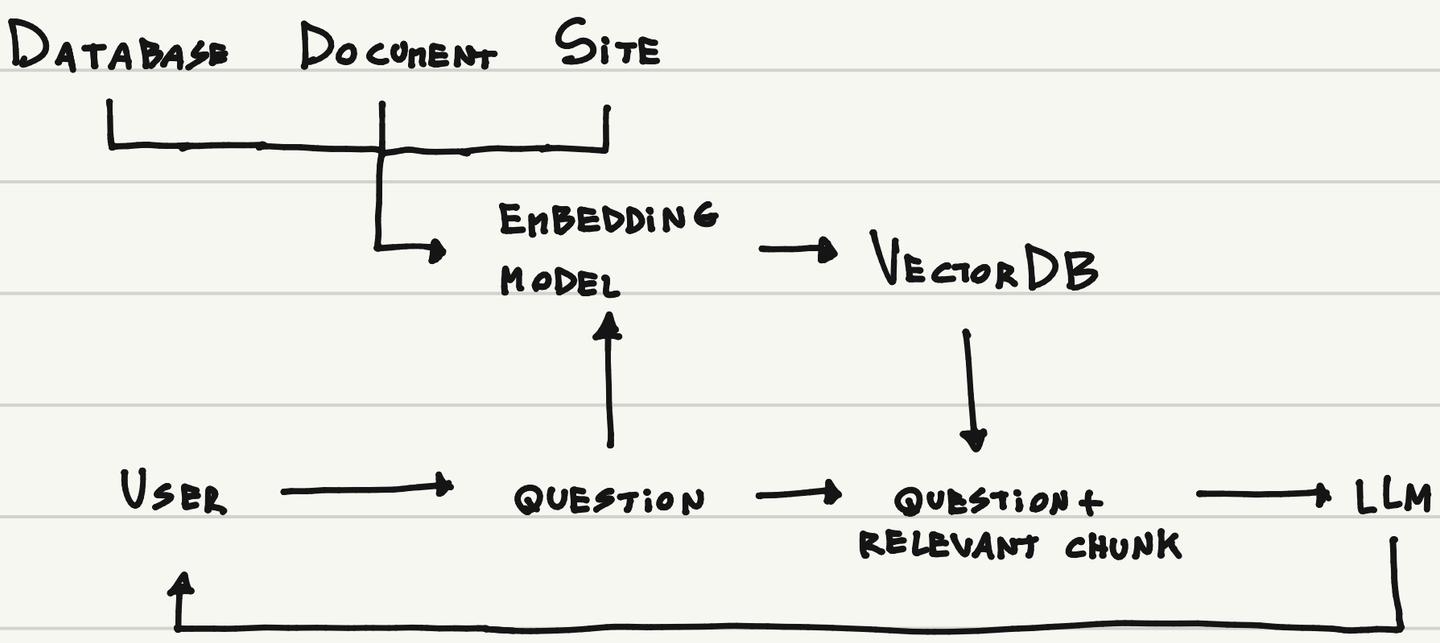


- **Retrieval Augmented Generation (RAG)**

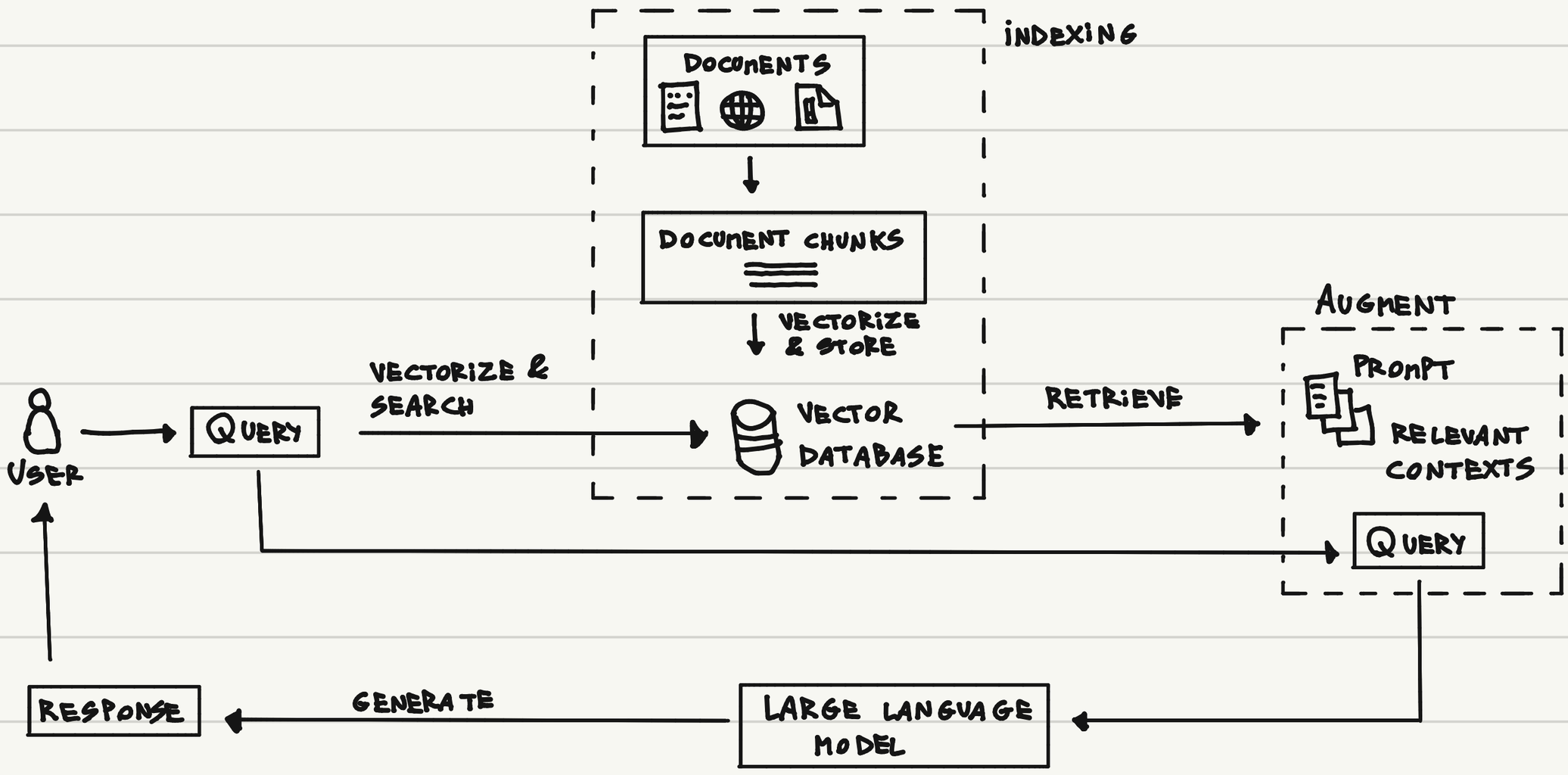


Encontrar o banco de dados vetorial mais adequado para a aplicação.

- Arquitetura simples de RAG



- Arquitetura simples de RAG - Visualização Expandida



- Separa um documento em pedaços (chunking), para só assim vetorizar.
- Bibliotecas úteis: LangChain e LlamaIndex. → Auxilian o processo de várias etapas de RAG.

- Arquitetura RAG (Exemplo do Code Review)

Código do Langchain → Loader → Documentos → Text Splitter → Chunks → Embeddings (HF) →

CHROMA (VECTOR DB) → RETRIEVER (MMR) → CONTEXTO RELEVANTE → PROMPT + CONTEXTO → GEMINI (LLM) → RESPOSTA

(A) LOADER: CARREGA OS ARQUIVOS .PY DO REPOSITÓRIO E TRANSFORMA EM DOCUMENTS.

(B) SPLITTER: DIVIDE OS ARQUIVOS EM CHUNKS DE ATÉ 2000 CARACTERES, COM SOBREPOSIÇÃO DE 200.

(C) EMBEDDINGS: TRANSFORMA CADA CHUNK EM UM VETOR NÚMERICO USANDO UM MODELO DO HUGGING FACE.

(D) CHROMA: ARMAZENA OS CHUNKS E SEUS EMBEDDINGS EM UM BANCO VETORIAL.

(E) RETRIEVER: RECEBE A PERGUNTA E BUSCA OS TRECHOS MAIS RELEVANTES. O MMR REDUZ REDUNDÂNCIA (K=8, CONSIDERANDO FETCH_K=20 CANDIDATOS).

(F) PROMPT: COMBINA A PERGUNTA DO USUÁRIO COM OS DOCUMENTOS RECUPERADOS EM {CONTEXT}.

(G) GEMINI: RECEBE O PROMPT+ CONTEXTO E GERA A RESPOSTA DA REVISÃO.

- PRINCIPAIS PROBLEMAS DA RAG

  ↳ UM TRADE-OFF A SER BALANCEADO.

  (A) CHUNK SIZE E TOP K. → RELATIVO AO PROBLEMA.

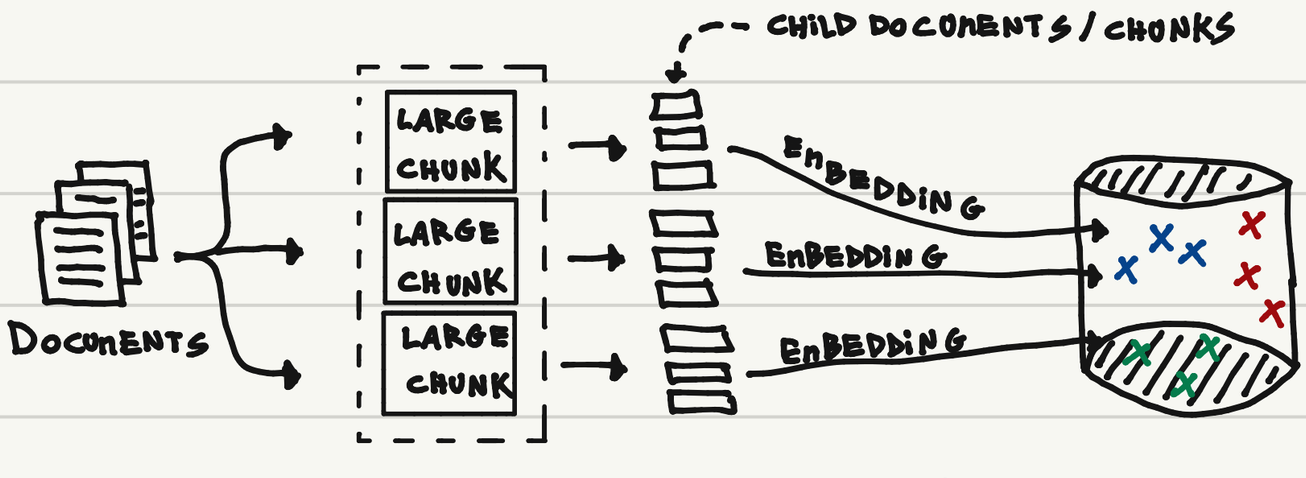
  (B) CONHECIMENTO DE MUNDO. → BASEADO NOS DADOS QUE O LLM FOI TREINADO.

  (C) PERDA DE INFORMAÇÃO. → BUSCA SEMÂNTICA NO VECTOR DB BASEADO NA QUALIDADE DO PROMPT E BASE DE DADOS VETORIAL UTILIZADA.

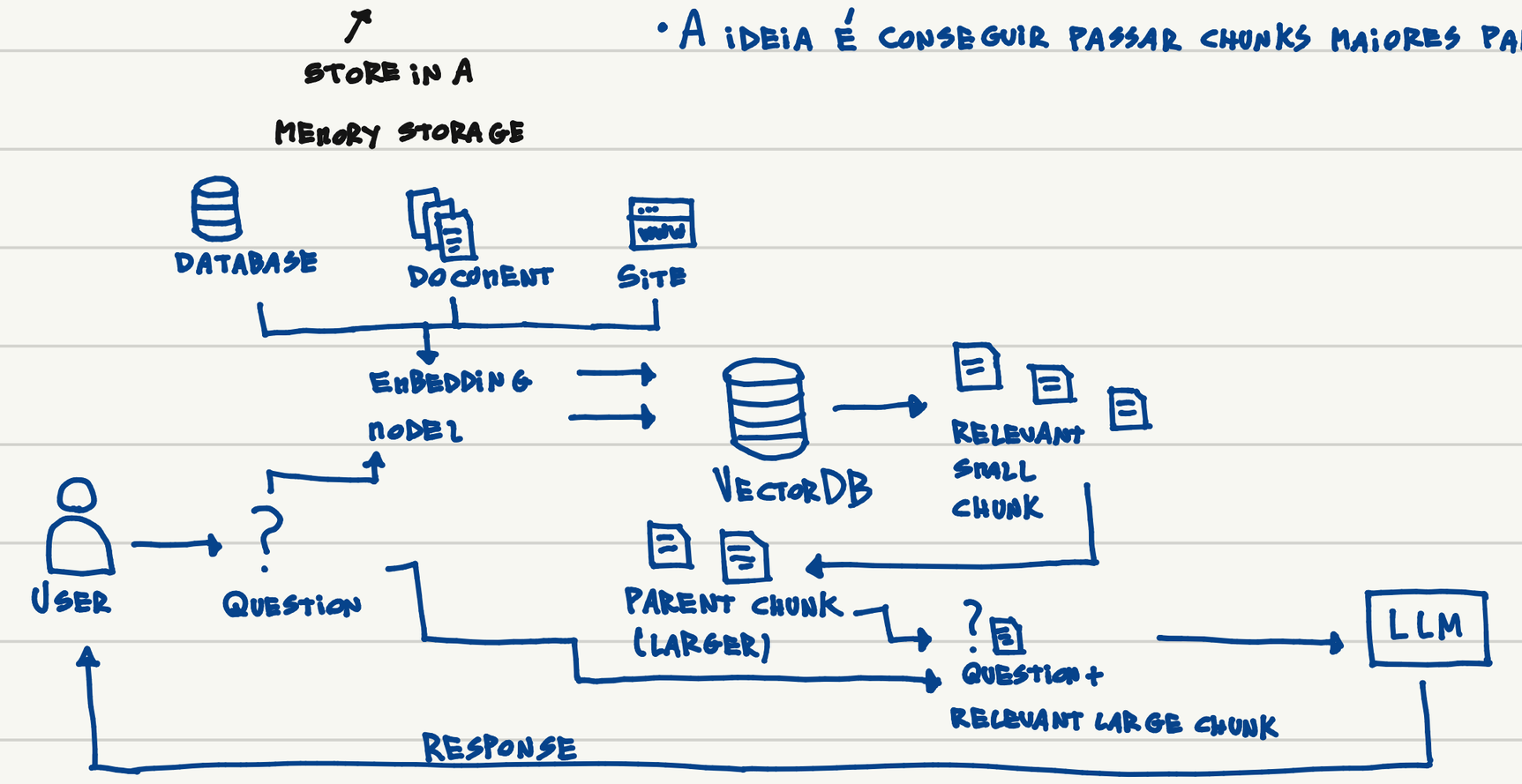
- NAIVE RAG OU BASIC RAG É AS SOLUÇÕES FEITAS ANTERIORMENTE, SENDO AS VERSÕES MIS BÁSICAS.

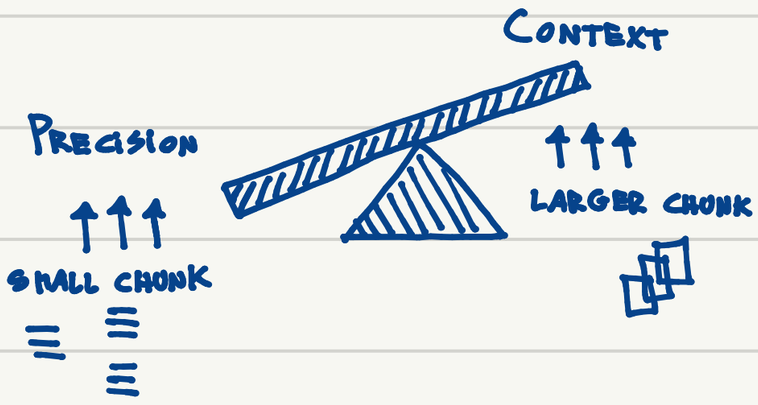
  ↳ TODA SOLUÇÃO QUE CONSISTE NA BUSCA DE CHUNKS DE FORMA SIMPLES.

- PARENT DOCUMENT RETRIEVER



- A IDEIA É CONSEGUIR PASSAR CHUNKS MAIORES PARA DAR MAIS CONTEXTO A LLM





1. SMALL CHUNKS (PEDAÇOS PEQUENOS) → FOCO EM PRECISÃO

OS PEDAÇOS PEQUENOS (GERALMENTE ENTRE 100 E 300 TOKENS) SÃO UTILIZADOS EXCLUSIVAMENTE NA ETAPA DE BUSCA VETORIAL.

↳ ALTA DENSIDADE SEMÂNTICA;

↳ REDUÇÃO DE RUÍDO;

↳ ALINHAMENTO COM A PERGUNTA;

2. LARGE CHUNKS (PEDAÇOS GRANDES) → CONTEXTO

OS PEDAÇOS GRANDES (GERALMENTE ENTRE 1000 E 4000 TOKENS OU O DOCUMENTO INTEIRO) ——→ CONTINUIDADE NARRATIVA;

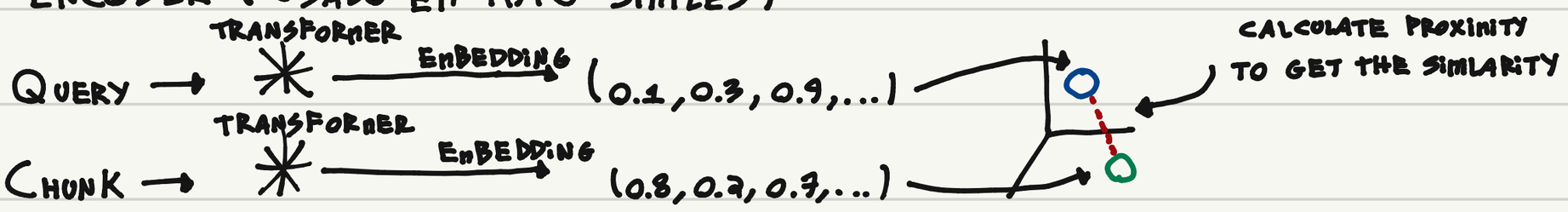
SÃO O QUE O LLM REALMENTE LÊ PARA GERAR A RESPOSTA. ↳ RESOLUÇÃO DE COERÊNCIA;

↳ MITIGAÇÃO DE ALUCINAÇÕES;

- PARA GARANTIR QUE A PERGUNTA E O CHUNK PARENTE CHEGUEM SINCRONIZADOS, É IMPORTANTE USAR UMA EXECUÇÃO EM PARALELO.

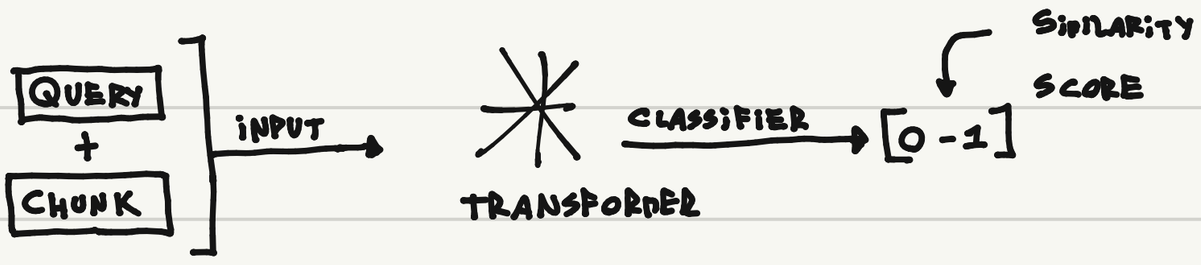
PERGUNTA DO USUÁRIO → RUNNABLE PARALLEL
- "QUESTION" → RUNNABLE PASSTHROUGH → "PERGUNTA ORIGINAL"
  - ↳ PASSA A ENTRADA ADIANTE SEM MODIFICAR.
- "CONTEXT" → RETRIEVER → DOCUMENTOS

- BI-ENCODER (USADO EM RAG SIMPLES)

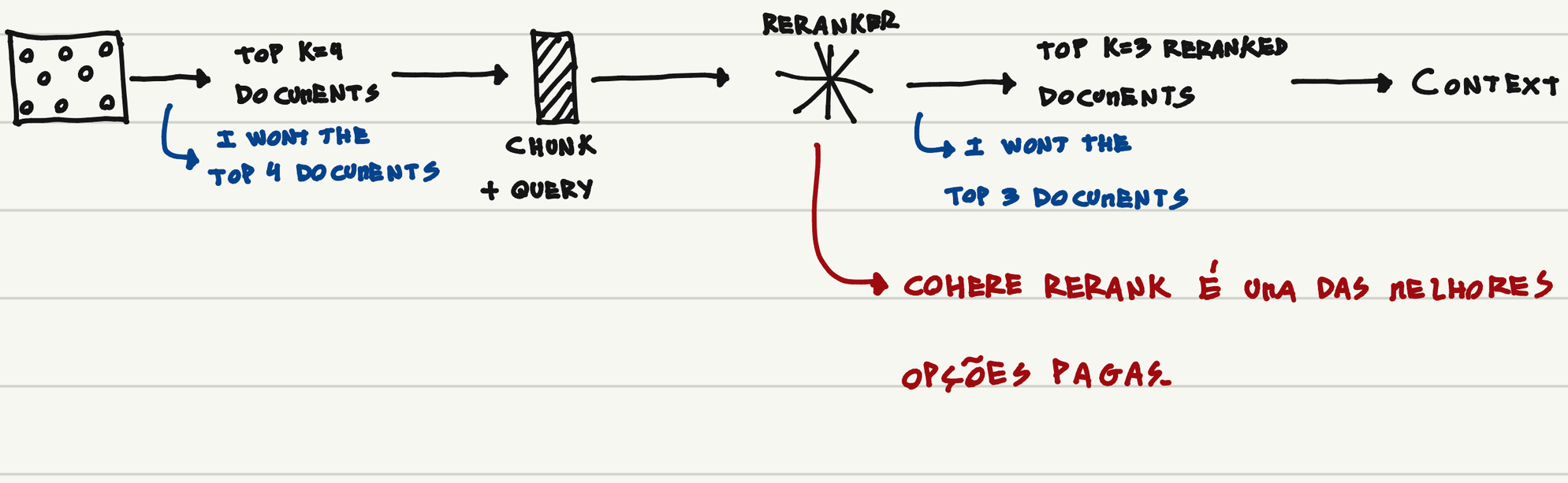


↳ OS EMBEDDINGS DA PERGUNTA E DOCUMENTO SÃO GERADOS SEPARADAMENTE.

- CROSS-ENCODER (USADO NO RERANKING)

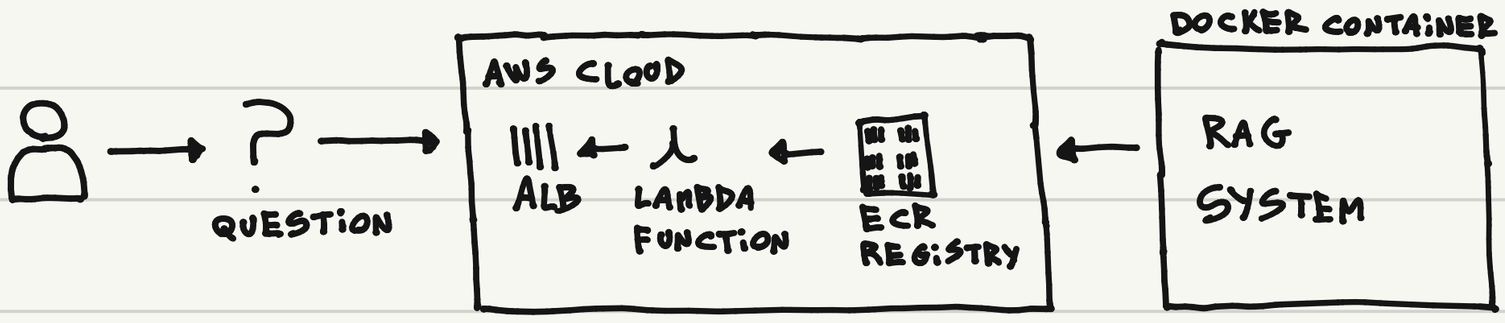


↳ AJUSTE FINO PARA ESCOLHER O EMBEDDING MAIS SIMILAR E REDUZIR OS CHUNKS



OPÇÕES PAGAS.

- DEPLOY DO RAG NA CLOUD



- ALB (Application Load Balancer) → NÃO EXECUTA CÓDIGO, APENAS DISTRIBUI O TRÁFEGO.
  - ALB É O PONTO DE ENTRADA DA APLICAÇÃO.
    - ↳ RECEBER REQUISIÇÕES HTTP/HTTPS DOS CLIENTES.
    - ↳ ENCERRAR TLS (HTTPS).
    - ↳ APLICAR REGRAS DE ROTEAMENTO.
    - ↳ ENCAMINHAR A REQUISIÇÃO PARA O BACKEND ADEQUADO.

- LAMBDA FUNCTION → APIs LEVES, AUTOMAÇÕES, PROCESSAMENTO DE ARQUIVOS

UMA AWS LAMBA FUNCTION EXECUTA CÓDIGO SOB DEMANDA (SERVERLESS).

↳ NÃO EXISTE SERVIDOR PARA ADMINISTRAR.

↳ É INICIADO APENAS QUANDO OCORRE UM EVENTO.

↳ COBRA APENAS PELO TEMPO DE EXECUÇÃO.

↳ ESCALA AUTOMATICAMENTE.

CLIENTE → ALB → LAMBDA → RESPOSTA

- ECR (ELASTIC CONTAINER REGISTRY)

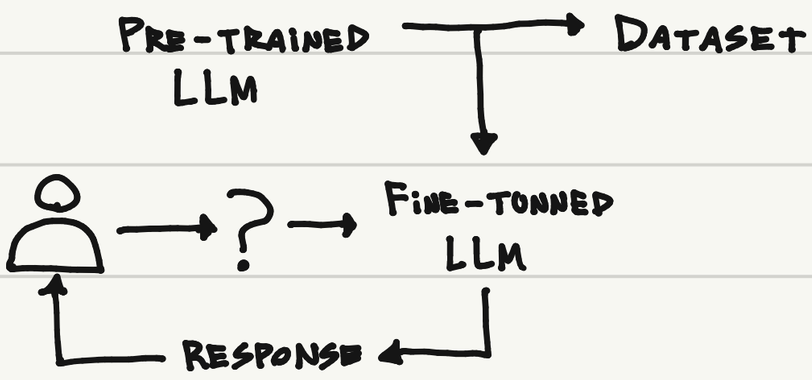
O ECR É UM REPOSITÓRIO DE IMAGENS DOCKER. SUA FUNÇÃO É ARMAZENAR IMAGENS QUE SERÃO UTILIZADAS POR SERVIÇOS DA AWS.

CÓDIGO → IMAGEM DOCKER → DOCKER PUSH → AMAZON ECR

- LLM'S (LARGE LANGUAGE MODEL) → FINE TUNNING

  - IDEAL PARA TASKS ESPECÍFICAS OU LINGUAGEM ESPECÍFICA.
    - ↳ TREINAR PARA TAREFAS ESPECÍFICAS OU PARA QUE O MODELO SE ESPECIALIZE EM ALGO (e.g. RADIOLOGIA)

→ NA MAIORIA DOS CASOS, LLM'S NÃO PRECISAM DE FINE-TUNNING.



- FINE-TUNING É O PROCESSO DE CONTINUAR O TREINAMENTO DE UMA LLM COM UM CONJUNTO DE DADOS ESPECÍFICO, AJUSTANDO SEUS PESOS INTERNOS PARA ESPECIALIZÁ-LO EM UM DOMÍNIO, TAREFA OU ESTILO DE RESPOSTA, DIFERENTEMENTE DO RAG, QUE MANTÉM OS PESOS INALTERADOS E FORNECE CONHECIMENTO POR MEIO DE RECUPERAÇÃO DE DOCUMENTOS EXTERNOS.

- É POSSÍVEL FAZER FINE-TUNING COM OS MODELOS DA OPENAI PORÉM É UM PROCEDIMENTO MUITO CARO.
- OUTRA OPÇÃO PARA FINE-TUNING É O AMAZON BEDROCK, É UMA CAMADA DE ABSTRAÇÃO SEM SERVIDOR FOCADA EM IA GENERATIVA POR API. É DIFERENTE DO AMAZON SAGEMAKER, QUE É UMA PLATAFORMA COMPLETA DE INFRAESTRUTURA PARA CRIAR, TREINAR E GERENCIAR QUALQUER MODELO DE APRENDIZADO DE MÁQUINA.
  - ↳ OS DADOS DE TREINAMENTO, TANTO PARA O AWS BEDROCK QUANTO PARA A OPENAI, SÃO ALIMENTADOS POR UM ARQUIVO JSONL. → OBSERVAR O PADRÃO DE JSON REQUERIDO PARA CADA UM.
    - ↳ ENVIO DOS DADOS E TREINAMENTO É FEITO POR API.

- NA MAIORIA DOS CASOS, SLM'S E OUTROS LM'S PRECISAM DE FINE-TUNING.
  - (A) GPT → 500B A 1T
  - (B) LLAMA 3 → 405B
  - (C) PHI-3 → 3B

- QUANDO A APLICAÇÃO NÃO É NECESSÁRIA USAR AMPLAMENTE OS RECURSOS DA GERAÇÃO DE TEXTO DOS LLM'S, AS VEZES MODELOS MAIS SIMPLES DE NLP RESOLVEM (e.g. DISCURSO LIMPO E DISCURSO DE ÓDIO).
  - ↳ BERT (BIDIRECIONAL TRANSFORMER FOR LANGUAGE UNDERSTAND).
    - ↳ ARTIGO "ATTENTION IS ALL YOU NEED".

- FINE-TUNING COM O HUGGING FACE

DATASET → TOKENIZER → TOKENIZAÇÃO → DATASET TOKENIZADO → MODELO PRÉ-TREINADO → TRAINER → TREINAMENTO → MODELO FINE-TUNED

- A TOKENIZAÇÃO OCORRE PORQUE O MODELO NÃO ENTENDE TEXTOS, APENAS IDS INTEIROS.

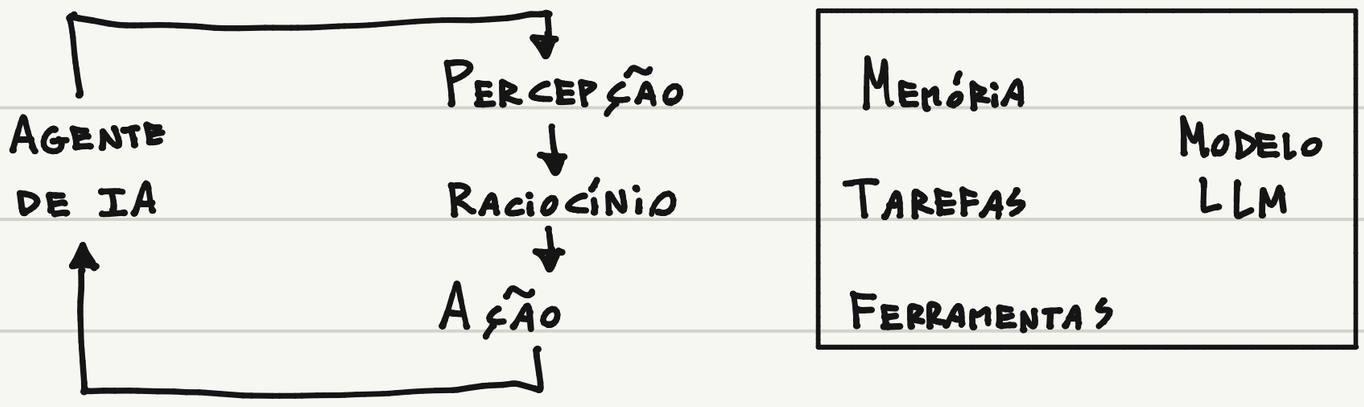
  "BOM DIA" → [2378, 319]

- O PROCESSO DE PADDING É PARA UNIFORMIZAR O TAMANHO DAS FRASES (A DIREITA OU A ESQUERDA COM PREENCHIMENTO DE ZEROS).

- A ARQUITETURA TRANSFORMER É UM TÉCNICA DE REDES NEURAIS BASEADA EM UM MECANISMO DE ATENÇÃO, PROJETADA PARA LIDAR COM SEQUÊNCIA DE DADOS, COMO TEXTO, SEM USAR ESTRUTURAS SEQUENCIAIS TRADICIONAIS.

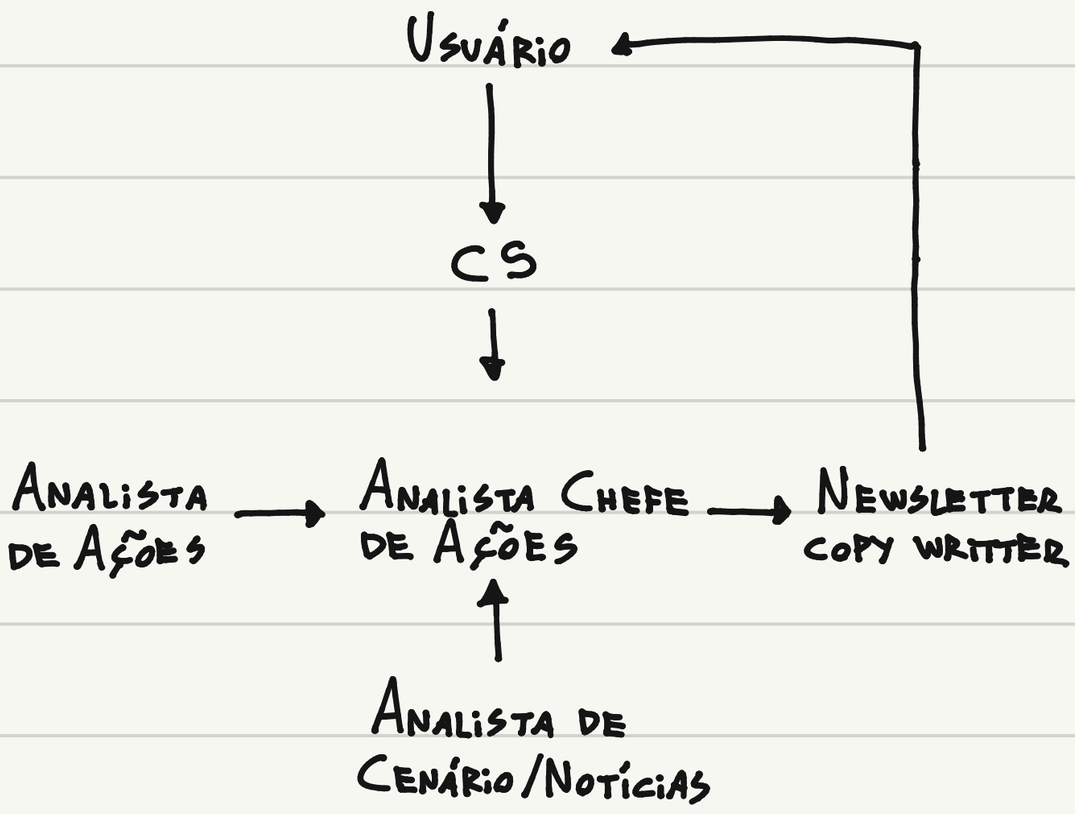
- AGENTES DE IA

AGENTES DE IA SÃO PROGRAMAS DE COMPUTADOR INTELIGENTES QUE TOMAM DECISÕES E REALIZAM TAREFAS DE FORMA AUTÔNOMA. ELES ENTENDEM METAS, USAM FERRAMENTAS E RESOLVEM PROBLEMAS SOZINHOS.



- Multi-Agentes de IA são sistemas formados por várias inteligências artificiais autônomas que colaboram entre si, dividindo tarefas e usando funções específicas para resolver problemas.
- CrewAI → ferramenta open-source para a criação dos agentes (origem brasileira).
- AutoGen → ferramenta open-source para a criação dos agentes da Microsoft.

- Exemplo de Agentes com CrewAI (Empresa de Análise de Ações)



- Ferramentas do Crew AI

(A) Agents: personas configuradas com restrições para atuar de forma especializada.

(B) Tools: interfaces de I/O com o mundo real (e.g. acessar internet e dados externos via API).

(C) Tasks: o que deve ser feito e cono deve ser entregue.

(D) Crew & Process: contêiner que agrupa os agentes e as tasks, definindo a topologia de execução do sistema.

- AutoGen → semelhante ao CrewAI | AutoGen Studio → Ferramenta Low-Code.

- Prompt Engineering

Prompt Engineering é o processo de estruturação de uma instrução que pode ser interpretada e compreendida por um modelo de IA generativa.

→ PESQUISAS DEMONSTRARAM QUE A ADIÇÃO DO TERMO "VAMOS PENSAR PASSO A PASSO" MELHORA OS RESULTADOS DO PROMPT.

↳ CHAIN-OF-THOUGHT

→ TÉCNICAS APLICADAS DIRETO NO CHAT.

(A) ZERO-SHOT PROMPTING: O PROMPT ZERO-SHOT INSTRUI DIRETAMENTE O MODELO A EXECUTAR UMA TAREFA SEM NENHUM EXEMPLO ADICIONAL PARA ORIENTAR.

(B) FEW-SHOT PROMPTING: PODE SER USADO COMO UMA TÉCNICA PARA HABILITAR O APRENDIZADO EM CONTEXTO, ONDE FORNECEMOS DEMONSTRAÇÕES NO PROMPT PARA DIRECIONAR O MODELO PARA UM MELHOR DESEMPENHO.

(C) CHAIN-OF-THOUGHTS PROMPTING: CoT PROMPTING PERMITE CAPACIDADES DE RACIOCÍNIO COMPLEXAS POR MEIO DE ETAPAS INTERMEDIÁRIAS DE RACIOCÍNIO.

(D) CIDI (CONTEXT, INSTRUCTIONS, DETAILS AND INPUT): FORNECENDO INFORMAÇÕES DE CONTEXTO, INSTRUÇÕES E DETALHES AJUDAM O MODELO A PERFORMAR MELHOR.

<CONTEXT>:

<INSTRUCTIONS>:

<DETAILS>:

<INPUT>:

→ TÉCNICAS APLICADAS PROGRAMATICAMENTE

(A) CIDI PODE SER APLICADO, AO INVÉS DE PASSAR DIRETAMENTE NO CHAT, PODE SER CONFIGURADO COMO INSTRUÇÃO NA PLATAFORMA DE LLM UTILIZADA.

(B) RETRIEVAL AUGMENTED GENERATION (RAG): FORNECENDO INFORMAÇÕES A UM AMPLO CONTEXTO COMO BASE DE CONHECIMENTO PARA A GERAÇÃO DA RESPOSTA, COMBINA UM COMPONENTE DE RECUPERAÇÃO DE INFORMAÇÕES COM UM MODELO GERADOR DE TEXTO.

(C) REACT PROMPTING (REASON AND ACT): NO MÉTODO REACT OS LLMS SÃO USADOS PARA GERAR RASTROS DE RACIOCÍNIO E AÇÕES ESPECÍFICAS DE TAREFAS DE FORMA INTERCALADA.

(D) TREE OF THOUGHS: TÉCNICA AVANÇADA DE IA QUE PERMITE AOS MODELOS DE LINGUAGEM EXPLORAR MÚLTIPLOS CAMINHOS DE RACIOCÍNIO, AVALIAR OPÇÕES E VOLTAR ATRÁS (BACKTRACKING) QUANDO NECESSÁRIO.